# ContextFlow: In-Context Flow Matching for Robot Manipulation

Jian Ding[1]*, Xianjie Dai[1]*, Roei Herzig[2], Nussair Hroub[1], Jinjie Mai[1], Dengxin Dai[3], Bernard Ghanem[1], and Mohamed Elhoseiny[1]†

[1] KAUST, Thuwal, Saudi Arabia; {jian.ding, dai.xianjie, nussair.hroub, jinjie.mai, bernard.ghanem, mohamed.elhoseiny}@kaust.edu.sa
[2] University of California, Berkeley, USA; roeiherz@gmail.com
[3] Zurich, Switzerland; ddx2004@gmail.com

The supplementary materials are organized as follows:

- **Additional Implementation Details** (Section S1), including implementation details of ContextFlow and baselines, and detailed dataset statistics for the real-world experiments.
- **Supplementary Experimental Results** (Section S2), including results with additional LIBERO task suites and splits, seen task configuration performance, extended real-world experiments, and visualizations.
- **Supplementary Analysis** (Section S3), including analysis of model behavior on the LIBERO-Object tasks suite and visualization and additional analysis of failure modes.
- **Discussion** (Section S4).

## S1 Additional Implementation Details

### S1.1 Implementation Details of ContextFlow

Observation images are encoded with SigLIP So400m/14 [9] at a resolution of 224×224, producing $N_{\text{img}} = 256$ visual tokens per camera view. Each multimodal context compressor uses $N_q = 32$ learnable query tokens; for image tokens, we allocate $N_q$ queries *per camera view*, yielding a total number of context tokens $N_{\text{context}} = N_{\text{cam}} \cdot N_q^{\text{img}} + N_q^{\text{state}} + N_q^{\text{action}}$, where $N_q^{\text{img}} = N_q^{\text{state}} = N_q^{\text{action}} = 32$. This results in $N_{\text{context}} = 128$ tokens for LIBERO experiments with $N_{\text{cam}} = 2$ and $N_{\text{context}} = 160$ tokens for ALOHA experiments with $N_{\text{cam}} = 3$. These context tokens are concatenated with the current observation tokens (256 image tokens) and text tokens (48) and fed to the context expert. We set the action horizon to $H = 50$ and parameterize noise action tokens following $\pi_0$ [2]. Both the context and action experts use multi-query attention and a Gemma-style [8] transformer architecture scaled down to our setting: the context expert uses embedding dimension $C_1 = 2048$, depth 18, and MLP dimension 4096, while the action expert uses embedding dimension $C_2 = 1024$ with the same depth and MLP dimension, initialized from $\pi_0$ [2] and fine-tuned using LoRA [4].

* Equal contribution.
† Corresponding author.

### S1.2 Implementation Details of Baselines

**ICRT** [3]. Following the original paper [3], we fine-tune ICRT with the *Llama2-Base* model, which is randomly initialized and fully fine-tuned. The sequence length for each sample is set to 512. We use an effective batch size of 8 samples, implemented as a per-device batch size of 2 with 4 gradient accumulation steps. We train the ICRT *Llama2-Base* model for 5 epochs on the LIBERO seen task configurations. All remaining fine-tuning hyperparameters follow the default settings provided in the original paper [3]. We also experimented with training for 10 epochs, but found that 5 epochs yielded better performance. In addition, we evaluated an *Llama2-7B* variant of ICRT, initialized from the pre-trained *Llama2-7B* language model and fine-tuned using LoRA. Consistent with the findings in the original paper [3], the *Llama2-7B* variant performed worse than the *Llama2-Base* model in our setting. During inference, we provide a single demonstration trajectory without subsampling as the prompt for each task.

**$\pi_0$ [2].** We follow the default settings of $\pi_0$ [2] in the official implementation [4] and fine-tune the $\pi_0$ base model for 30K iterations with a batch size of 32 without LoRA [4] and EMA on seen task configurations. Note that we disable *extra delta transform* since the LIBERO action is of OSC-POSE and already delta.

**$\pi_0$ [2] FT.** For one-shot fine-tuning on each unseen task, we start from the 30k-iteration checkpoint and fine-tune on a single demonstration from that unseen task for 100 iterations, a batch size of 2 and a constant learning rate of $2.5 \times 10^{-6}$. We repeat this procedure separately for each unseen task. The reported performance for each task therefore reflects traditional one-shot learning that requires updating model weights. This contrast highlights the advantage of our in-context learning approach, which achieves comparable or superior performance without any task-specific parameter updates.

**OpenVLA-OFT**. We follow the default training settings of LIBERO from the official implementation[5], with the exception that we reduce the number of GPUs to four, limit the total iterations to 20,000, and use a per-GPU batch size of 8. These modifications are necessitated by our computational constraints and the exceptionally large size of the model (7B parameters). However, under this reduced configuration, we observe sub-optimal performance on seen task configurations compared to the results reported in the original OpenVLA-OFT work, which is likely attributable to our lower computational budget and modified training parameters.

**ContextAR**. We build our in-context learning implementation on the $\pi_0$-FAST [7] base model. Recall that $\pi_0$-FAST discretizes the normalized proprioceptive state into 256 bins and concatenates its string representation with the language instruction as part of the prefix, which is then tokenized by the PaliGemma [1] tokenizer. The action sequence is tokenized separately by the FAST tokenizer as a postfix and mapped to the final indices of the PaliGemma vocabulary. To

[4] `https://github.com/Physical-Intelligence/openpi`
[5] `https://github.com/moojink/openvla-oft/blob/main/LIBERO.md`

**Table S1: Seen and unseen task configurations in the real-world aloha experiments.**

| | |
|---|---|
| **Pick & Place**: "pick up ⟨object⟩ and place it in the basket with ⟨left/right⟩ hand" | |
| **Seen (14)** | ⟨left⟩: apple, corn, gray milk, carrot, chips ⟨right⟩: pear, orange juice, gray milk, cucumber, corn, red apple, chips, blue milk, carrot |
| **Unseen (4)** | ⟨left⟩: pear, orange juice ⟨right⟩: kiwi, banana |
| **Pen Uncap**: "pick up ⟨pen⟩ with ⟨left/right⟩ hand, grasp the cap with another hand and uncap it" | |
| **Seen (6)** | ⟨left & right⟩: gray pen, blue pen v2 ⟨right⟩: red pen ⟨right⟩: blue pen |
| **Unseen (1)** | ⟨left⟩: red pen |
| **Put Egg in Box**: "pick up the ⟨object⟩ with right hand, place it in the box, and close the box" | |
| **Seen (1)** | white egg **Unseen (1)** red egg |
| **Extra Bimanual Task configurations** | |
| **Seen (4)** | handover, cup stack, stir, water wipe |

avoid token explosion in the in-context demonstration, we implement a projection layer that maps each discrete in-context state into a single 2048-dimensional (for base model) continuous token. A parallel projection layer is applied to in-context actions. This design allows us to preserve the original training input format while effectively leveraging the pretrained model without incurring substantial architectural changes.

### S1.3 Details of Real-World Experiments

We conducted real-world experiments on a mobile ALOHA robot platform. The scene consists of a table and toy objects (e.g., toy fruits, vegetables, eggs) and real pens. The setup includes one high camera and two wrist-mounted cameras, and demonstrations are collected via human teleoperation. We collected a dataset that includes both single arm task configurations and fine-grained bimanual manipulation task configurations. Detailed task configurations are shown in Tab. S1. The statistics of training data are shown in Tab. S2. Pick Up and Place tasks follow the instruction template `‘‘Pick up the <object> and place it in the basket with the <left/right> hand’’`. There are two possible basket locations: (i) in front of the midpoint between the two arms, in which case the task is executed with the right hand, and (ii) in front of the left arm, in which case the task is executed with the left hand. Across task configurations, we vary the positions of both the objects and the basket to increase task diversity. Pen Uncap tasks follow the instruction template `‘‘Pick up the <pen> with the <left/right> hand, grasp the cap with the other hand and uncap it’’`. There are 4 different kinds of pens (gray, red, blue, and blue v2), varying in shape, color, and force properties required for uncapping. For each pen, there are two motion trajectory variants depending on which hand is used to pick up the pen first; the other hand then grasps and removes the cap, making this a bimanual coordination task. Put Egg in Box tasks follow the instruction template `‘‘Pick up the <object> with right hand, place it in the`

**Table S2: Training data statistics for seen task configurations.**

| Task Suite | Task configurations | Episodes | Avg Length |
|---|---|---|---|
| Pick & Place | apple / ⟨left⟩ | 25 | 200 |
| | corn / ⟨left⟩ | 25 | 200 |
| | gray milk / ⟨left⟩ | 25 | 200 |
| | carrot / ⟨left⟩ | 25 | 200 |
| | chips / ⟨left⟩ | 25 | 200 |
| | pear / ⟨right⟩ | 50 | 250 |
| | orange juice / ⟨right⟩ | 25 | 240 |
| | gray milk / ⟨right⟩ | 26 | 204 |
| | cucumber / ⟨right⟩ | 50 | 250 |
| | corn / ⟨right⟩ | 50 | 246 |
| | red apple / ⟨right⟩ | 25 | 200 |
| | chips / ⟨right⟩ | 25 | 200 |
| | blue milk / ⟨right⟩ | 76 | 267 |
| | carrot / ⟨right⟩ | 75 | 265 |
| | *Subtotal* | *527* | *235* |
| Pen Uncap | gray pen / ⟨left⟩ | 143 | 604 |
| | gray pen / ⟨right⟩ | 51 | 500 |
| | red pen / ⟨right⟩ | 69 | 497 |
| | blue pen / ⟨right⟩ | 25 | 500 |
| | blue pen v2 / ⟨left⟩ | 25 | 500 |
| | blue pen v2 / ⟨right⟩ | 25 | 500 |
| | *Subtotal* | *338* | *543* |
| Put Egg in Box | white egg | 199 | 735 |
| | *Subtotal* | *199* | *735* |
| Extra Bimanual (train only) | handover | 104 | 642 |
| | cup stack | 50 | 400 |
| | stir | 50 | 350 |
| | water wipe | 50 | 500 |
| | *Subtotal* | *254* | *509* |
| **Total (3 main suites)** | | **1,064** | – |
| **Total (all)** | | **1,318** | – |

`box, and close box''`. This is another fine-grained bimanual task that requires the robot to pick up the egg, carefully place it into the box, and then close the lid using both hands. Training data includes putting a white egg in the box and closing it, while the unseen test task uses a red egg to evaluate generalization to novel object appearance. In addition, we include 4 extra bimanual tasks (handover, cup stack, stir, and water wipe) with 254 demonstrations for training only. These tasks further enrich the training distribution with diverse bimanual manipulation behaviors.

## S2 Supplementary Experimental Results

### S2.1 Results on unseen and seen tasks across all task suites

We compare our method against $\pi_0$ [2] and ICRT [3] on all task configurations, including both seen and unseen task configurations across four task suites, as

**Table S3: Success rates on seen and unseen task configurations across all task suites in LIBERO.**

| Model | Seen Task Configurations | | | | | Unseen Task Configurations | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Spatial | Object | Goal | 10 | Avg. | Spatial | Object | Goal | 10 | Avg. |
| $\pi_0$ [2] | **96.0** | 97.0 | **94.0** | **87.0** | **93.0** | 26.0 | 63.0 | 0.0 | 0.0 | 22.0 |
| ICRT [3] | 85.0 | **99.0** | 90.0 | 65.0 | 85.0 | 27.0 | 50.0 | **1.0** | 0.0 | 20.0 |
| ContextFlow (Ours) | 91.0 | 97.0 | 90.0 | 82.0 | 90.0 | **64.0** | **83.0** | 0.0 | 0.0 | **37.0** |

shown in Tab. S3. Although ContextFlow is designed for *unseen* task configurations, it achieves performance comparable to $\pi_0$ on the seen task configurations. We observe that all models struggle on the Goal and Long-Horizon-10 *unseen* task configurations, likely due to the larger task variations in these suites. These results highlight the limitations of current in-context imitation learning methods.

### S2.2 Different Train-Test Splits on LIBERO

We further evaluate the performance of models under different train-test task splits on LIBERO. Details of all task indexes and corresponding language descriptions are available at huggingface[6]. We constructed three versions of train-test splits, with the *unseen* task configurations for each version listed in Table S4. The models are only trained with seen task configurations, and tested on unseen task configurations. The corresponding results are presented in Table S5. As shown, ContextFlow consistently surpasses the in-context baseline ICRT on unseen task configurations for every data split version and achieves the highest average success rate across all three task splits.

### S2.3 Comparison on Real-world experiments

We additionally evaluate $\pi_0$ [2] in the real-world experiments. As shown in Tab. S7, ContextFlow demonstrates stronger generalization to unseen task configurations compared to both baselines. We observe that success rates are influenced by both the intrinsic task difficulty and the gap between seen and unseen task configurations. For single-armed pick-and-place tasks, which are short-horizon and relatively simple, the unseen test objects and positions differ substantially from those seen during training. In this setting, $\pi_0$ achieves moderate success rates but remains significantly below ContextFlow. For the put-egg-in-box task, although it is a long-horizon fine-grained bimanual task, the difference between seen and unseen configurations is minimal (white egg vs. red egg), and $\pi_0$ performs well. In contrast, the uncap-red-pen task combines fine-grained bimanual coordination with a larger task configuration gap between seen and unseen. Here, $\pi_0$ fails entirely, while ContextFlow still maintains a reasonable success rate.

[6] `https://huggingface.co/datasets/physical-intelligence/libero/blob/main/meta/tasks.jsonl`

**Table S4: Unseen task configurations across different train-test split versions.**

| Versions | Task Suites | Task Indexes | Language Descriptions |
|---|---|---|---|
| v1 | Spatial | 35 | Pick up the black bowl on the cookie box and place it on the plate |
| | | 36 | Pick up the black bowl next to the plate and place it on the plate |
| | Object | 25 | Pick up the milk and place it in the basket |
| | | 28 | Pick up the tomato sauce and place it in the basket |
| | Goal | 10 | Put the bowl on the plate |
| | | 17 | Put the bowl on the stove |
| | Long-horizon 10 | 1 | Put the white mug on the plate and put the chocolate pudding to the right of the plate |
| | | 5 | Put both the alphabet soup and the tomato sauce in the basket |
| v2 | Spatial | 33 | Pick up the black bowl on the stove and place it on the plate |
| | | 37 | Pick up the black bowl next to the ramekin and place it on the plate |
| | Object | 21 | Pick up the ketchup and place it in the basket |
| | | 23 | Pick up the BBQ sauce and place it in the basket |
| | Goal | 11 | Put the wine bottle on the rack |
| | | 13 | Put the cream cheese in the bowl |
| | Long-horizon 10 | 3 | Turn on the stove and put the moka pot on it |
| | | 7 | Put both the cream cheese box and the butter in the basket |
| v3 | Spatial | 31 | Pick up the black bowl in the top drawer of the wooden cabinet and place it on the plate |
| | | 32 | Pick up the black bowl on the ramekin and place it on the plate |
| | Object | 29 | Pick up the chocolate pudding and place it in the basket |
| | | 20 | Pick up the orange juice and place it in the basket |
| | Goal | 11 | Put the wine bottle on the rack |
| | | 18 | Put the bowl on top of the cabinet |
| | Long-horizon 10 | 2 | Put the yellow and white mug in the microwave and close it |
| | | 8 | Put the black bowl in the bottom drawer of the cabinet and close it |

### S2.4 Visualization of ContextFlow Rollout

In Fig. S1 and Fig. S3, we visualize an in-context demonstration and the corresponding ContextFlow rollout in the real world and in simulation, respectively. Note that the initial states of the in-context demonstration and the test scene are similar but not identical. For example, in the pick-and-place pear task, the pear location differs slightly, and in the pen uncap task, the pen location and orientation vary between the demonstration and test scene. Additional rollout examples are provided in the supplementary videos.

**Robustness to discrepancies between in-context demonstrations and test scenes.** In practice, the initial state of the test scene is rarely identical to that of the in-context demonstration. We further evaluate ContextFlow's robustness by intentionally increasing such discrepancies, with qualitative examples shown in Fig. S2. The first example involves a spatial difference, where the banana is placed at a significantly different location from the demonstration. The second example involves an object-level difference, where the banana is cut in half and only one piece is used for testing. In both cases, ContextFlow successfully completes the task despite the discrepancies, demonstrating its ability to generalize across variations in initial conditions.

**Table S5: Success rates across different LIBERO splits.** We report Spatial, Object, and combined averages for Splitv1, Splitv2, and Splitv3, along with the overall Total Average across splits.

| Model | Splitv1 | | | Splitv2 | | | Splitv3 | | | Total Avg. |
|---|---|---|---|---|---|---|---|---|---|---|
| | Spatial | Object | Avg. | Spatial | Object | Avg. | Spatial | Object | Avg. | |
| $\pi_0$ [2] | 26.0 | 63.0 | 44.5 | 0 | 44.0 | 22.0 | 10.0 | 48.0 | 29.0 | 31.8 |
| ICRT [3] | 27.0 | 50.0 | 38.5 | 1.0 | 48.0 | 24.5 | 1.0 | **100.0** | 50.5 | 37.8 |
| ContextFlow (Ours) | **64.0** | **83.0** | **73.5** | **2.0** | **58.0** | **30.0** | **52.0** | 62.0 | **57.0** | **53.5** |

### S2.5 Computation cost for training and inference

We reported A100 GPU hours and inference throughput (Hz.) in Tab. S6. We followed OpenVLA-OFT [6] for the computation of throughput. We can see that flow matching models are usually faster than the autoregressive models due to the prediction of action chunk in parallel. ContextFlow-Plain is fast because it uses only 2 images. However, increasing the number of image tokens in ContextFlow-Plain leads to degraded performance. This suggests that without an image compressor (used in ContextFlow), directly attending to a large number of tokens is not only computationally expensive but also makes it difficult to attend to the relevant tokens for in-context imitation learning.

**Table S6:** Training and inference computational costs of different models. We report the training time in A100 GPU hours and the inference throughput (Hz) measured on a single A100 GPU.

| Model | Training (Hours) ↓ | Inference (Hz) ↑ |
|---|---|---|
| OpenVLA-OFT [6] | 30 | 77.8 |
| $\pi_0$ [2] | 21 | 569.0 |
| ICRT [3] | 16 | 44.3 |
| ContextAR | 65 | 3.7 |
| ContextFlow-Plain | 21 | 310.2 |
| ContextFlow | 40 | 130.6 |

## S3 Supplementary Analysis

### S3.1 Model Behavior Analysis on LIBERO-Object

We approximate each 3D grasp position as the end-effector position at the time step with the largest frame-to-frame change in gripper openness. The XY distribution of these grasp positions across all LIBERO Object episodes exhibits two clear spatial clusters (Fig. S6 (a)), so we further analyze the Z-dimension within each cluster. Specifically, we examine the Z-coordinate distributions of grasp positions for seen and unseen expert demonstrations (with green and blue background, respectively) and for inference rollouts (RO) of ICRT and ContextFlow

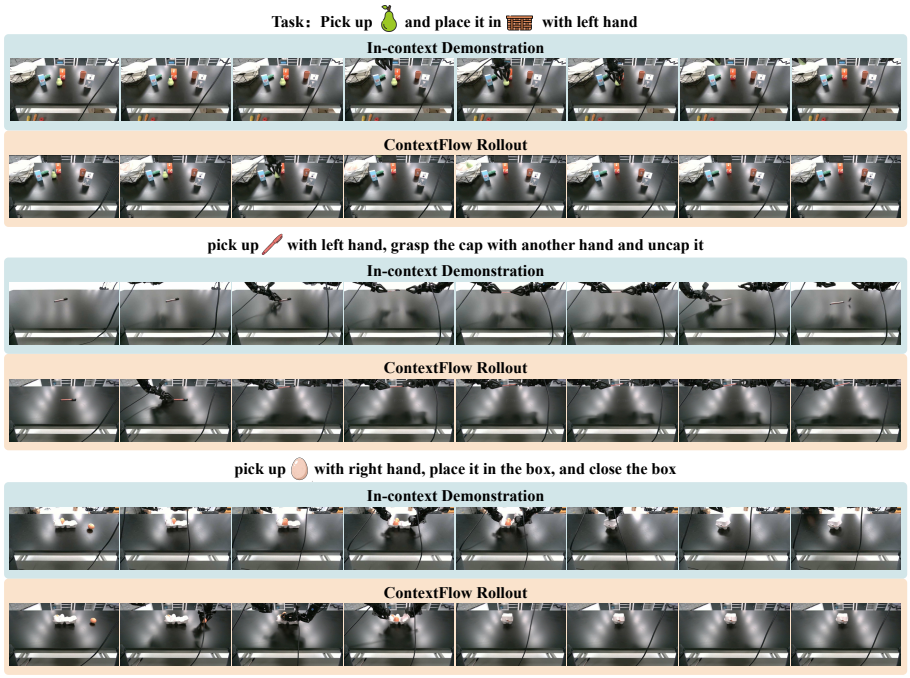


**Fig. S1: Rollout of ContextFlow in real-world experiments.** We show the in-context demonstrations and the corresponding ContextFlow rollouts from high camera.

(with orange background). In the cluster shown in Fig. S6 (b), the average grasp height in the *seen* task configurations is close to that of the *unseen tomato sauce* object, and the grasp-height distributions of both ICRT RO and ContextFlow RO align well with the expert demonstrations, leading to comparatively high success rates. In contrast, in the cluster shown in Fig. S6 (c), most training objects (such as ketchup and chocolate) have relatively low grasp points, lowering the average grasp height, whereas the unseen *milk* object is positioned noticeably higher. This mismatch between the seen-task average and the *milk* object makes the task challenging: ICRT achieves a 4.0% success rate, while ContextFlow reaches 76%. Consistent with this difference, the ICRT grasp-height distribution remains concentrated around the seen-task average, which is associated with a higher rate of grasp failures, whereas ContextFlow shifts its grasp heights upward and better aligns with the unseen object. Overall, this analysis indicates that grasp-height variation within a spatial cluster can substantially influence performance, and that ContextFlow adjusts its grasp height more flexibly to accommodate unseen object geometry under these conditions.

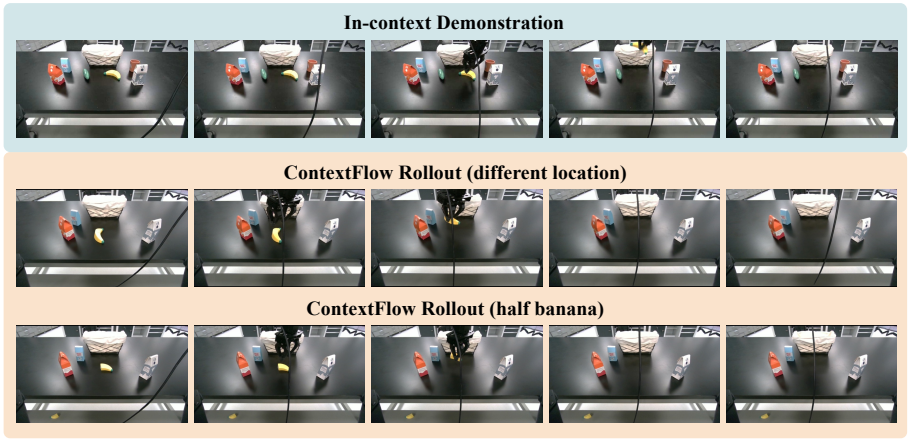


**Fig. S2: Generalization of ContextFlow under distribution shifts from the in-context demonstration.** The first row shows the in-context demonstration. The second row shows the rollout where the banana is placed at a different location and orientation. The third row shows the rollout where the banana is cut in half, introducing a novel object appearance.

### S3.2 Additional Failure Mode Analysis

**Visualization of Failure Modes.** In Fig 5. *Failure mode analysis* in the main paper, we provide failure mode analysis of $S_2$ "Pick up black bowl next to the plate and place it on the plate task". We further visualize three types of failure modes in Fig. S5. The first is *mis-identification*, where the model acts on the wrong object despite being given the correct prompt. This often arises when multiple visually or semantically similar objects are in close proximity, creating ambiguity in target selection. For example, the gripper tries to grasp the plate instead of the bowl. The second failure mode is *grasp failure*, where the model identifies the correct object but the gripper fails to establish a successful grasp. This type of failure may be related to inaccurate action prediction. The third is *place failure*, where the robot grasps the object correctly but fails to place it at the target location.

**Failure Mode Analysis on Task $O_1$.** We further analyze the failure modes of task $O_1$ "Pick up the milk and place it in the basket", as shown in Fig. S4. With the in-context prompt, both ICRT and ContextFlow reliably identify the correct target object, indicating that mis-identification is not a primary source of error. Most remaining failures occur during the grasping stage for both methods, with ContextFlow exhibiting a higher proportion of successful grasps. For ICRT, additional errors also appear in the placement stage, where some rollouts fail to place the object into the basket after a successful grasp.

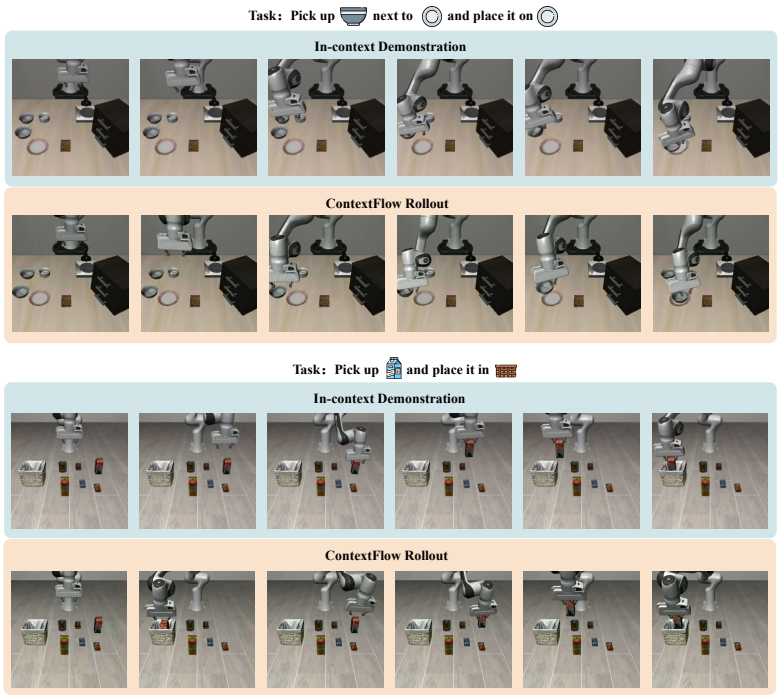


**Fig. S3: Rollout of ContextFlow in LIBERO simulation.** We show the in-context demonstrations and the corresponding ContextFlow rollouts from both agent and wrist views. "Pick up black bowl next to the plate and place it on the plate" belongs to LIBERO Spatial task suite and "Pick up the milk and place it in the basket" belongs to LIBERO Object task suite.

### S3.3 Attention in Image Compressor

To understand what the image compressor learns, we have visualized the attention of learnable queries for task "pick up the black bowl on the cookie box and place it on the plate" in main paper. Here, we provide visualization for other unseen task configurations in Fig. S7. Similarly, we notice attention changes along task phases.

## S4 Discussion

Our experiments show that flow-matching policies achieve higher accuracy and more robust generalization than autoregressive models in the in-context imitation setting, particularly on unseen task configurations where distribution shift exacerbates compounding prediction errors. By predicting continuous action chunks in parallel, flow matching avoids the strict token-by-token dependency of autoregressive models and thus reduces error accumulation over long horizons.

**Table S7: Success rates on unseen task configurations on a real-world robot.** See Sec. S1.3 for task instruction templates. Elements in different task configurations are listed in the table. Note that the pen uncap task here uses the left hand to pick up the pen (i.e., "pick up the red pen with left hand, grasp the cap with the other hand and uncap it".)

| Model | Single-armed Pick-and-place tasks | | | | Bimanual tasks | | Avg. |
|---|---|---|---|---|---|---|---|
| | Left hand | | Right hand | | Uncap | Put in box | |
| | Pear | Orange | Banana | Kiwi | Red Pen | Red Egg | |
| ICRT [3] | **2/10** | 0/10 | 0/10 | 0/10 | 0/10 | 0/10 | 3.3 |
| $\pi_0$ [2] | **2/10** | 2/10 | 3/10 | 2/10 | 0/10 | **7/10** | 26.7 |
| ContextFlow (Ours) | **2/10** | **5/10** | **4/10** | **4/10** | **4/10** | 6/10 | **41.7** |

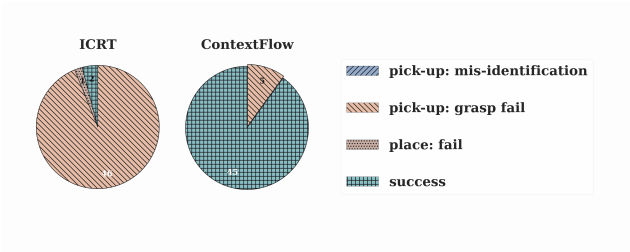


**Fig. S4: Failure mode analysis** of $O_1$ "Pick up the milk and place it in the basket". With in-context prompting, both methods correctly identify the target object. Most remaining errors arise at the grasp stage for both models, while ContextFlow achieves a higher proportion of successful rollouts under this setting.

On the other hand, recent work such as $\pi_0$-Fast [7] suggests that autoregressive heads can be trained more efficiently than flow-based ones, benefiting from a simpler next-token prediction objective and faster convergence. This trade-off points to a promising hybrid direction: first train an in-context imitation model with an autoregressive objective for efficiency, then fine-tune a flow-matching head for improved robustness and accuracy, such as $\pi_{0.5}$ [5]. Exploring such staged or joint AR–flow training schemes is an interesting avenue for future work.

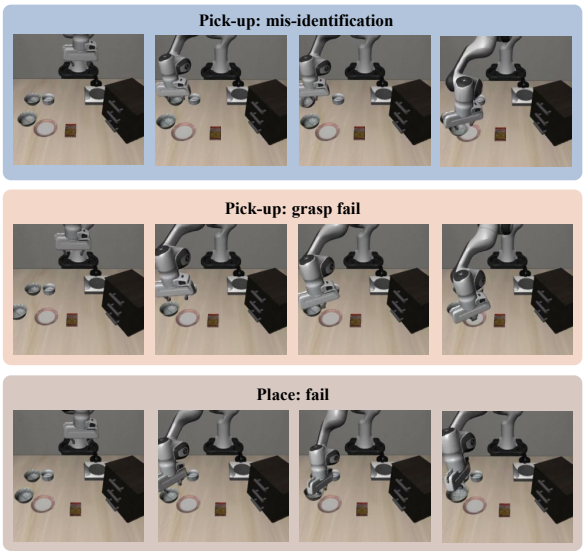


**Fig. S5: Visualization of three types of failure modes.** The task for the visualization is "pick up the black bowl next to the plate and place it on the plate."

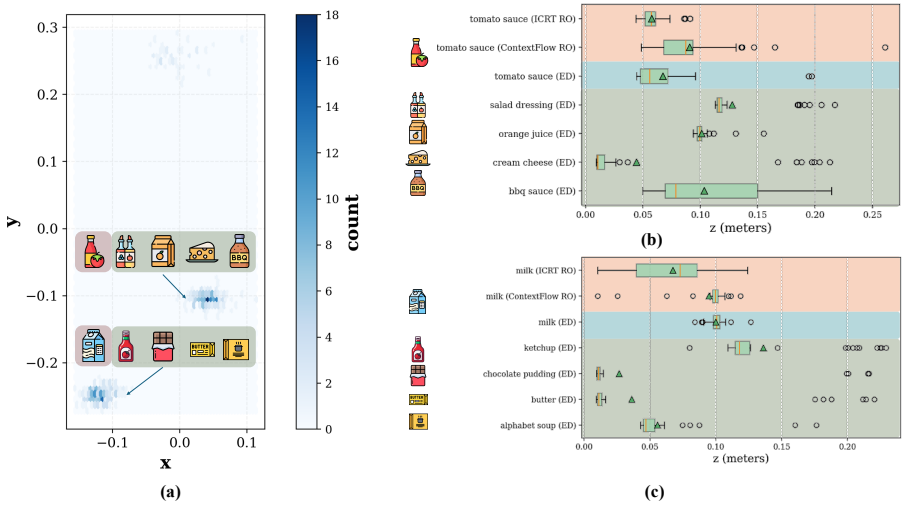


**Fig. S6: Grasp position analysis of the LIBERO-object task suite.** (a) XY distribution of grasp positions across all demonstrations, revealing two distinct spatial clusters. (b) Z-coordinate distributions for task configurations belonging to the cluster that contains the unseen *tomato sauce* task, whose object height is close to the training average. (c) Z-coordinate distributions for task configurations belonging to the cluster that contains the unseen *milk* task, whose object height is notably higher than the training average. In (b) and (c), green triangles mark the mean Z-coordinate across multiple demonstrations, yellow vertical lines denote the median, and dots indicate outliers. Tasks configurations in the lower green region correspond to *seen* expert demonstrations (ED); the middle blue region corresponds to *unseen* task configurations in LIBERO-Object; and the upper red region corresponds to ICRT and ContextFlow inference rollouts (RO). We observe that when the height of an *unseen* object differs from the training average, ICRT rollouts tend to follow the average grasp position and fail to match the unseen object's height, whereas our proposed ContextFlow generalizes well and aligns its grasp height with the unseen objects.

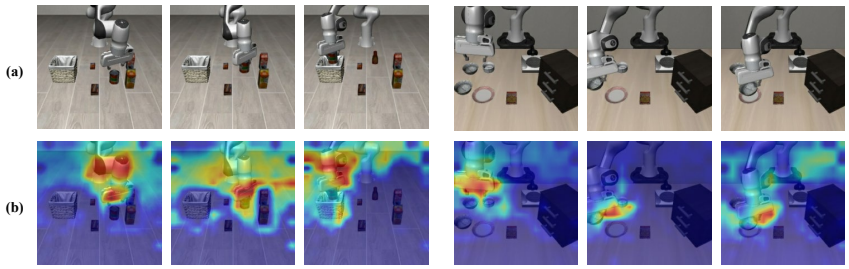


**Fig. S7: Visualization of attention in image compressor on unseen task configurations.** (a) Frames in In-context Demonstration (b) Image compressor attention. The left shows the task "pick up the tomato sauce and place it in the basket", while the right shows the task "pick up the black bowl next to the plate and place it on the plate".

## References

1. Beyer, L., Steiner, A., Pinto, A.S., Kolesnikov, A., Wang, X., Salz, D., Neumann, M., Alabdulmohsin, I., Tschannen, M., Bugliarello, E., et al.: Paligemma: A versatile 3b vlm for transfer. arXiv preprint arXiv:2407.07726 (2024)
2. Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., et al.: $\pi_0$: A vision-language-action flow model for general robot control. arXiv preprint arXiv:2410.24164 (2024)
3. Fu, L., Huang, H., Datta, G., Chen, L.Y., Panitch, W.C.H., Liu, F., Li, H., Goldberg, K.: In-context imitation learning via next-token prediction. arXiv:2408.15980 (2024)
4. Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al.: Lora: Low-rank adaptation of large language models. ICLR **1**(2), 3 (2022)
5. Intelligence, P., Black, K., Brown, N., Darpinian, J., Dhabalia, K., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., et al.: $\pi_{0.5}$: A vision-language-action model with open-world generalization. arXiv preprint arXiv:2504.16054 (2025)
6. Kim, M.J., Finn, C., Liang, P.: Fine-tuning vision-language-action models: Optimizing speed and success. arXiv preprint arXiv:2502.19645 (2025)
7. Pertsch, K., Stachowicz, K., Ichter, B., Driess, D., Nair, S., Vuong, Q., Mees, O., Finn, C., Levine, S.: Fast: Efficient action tokenization for vision-language-action models. arXiv preprint arXiv:2501.09747 (2025)
8. Team, G., Mesnard, T., Hardin, C., Dadashi, R., Bhupatiraju, S., Pathak, S., Sifre, L., Rivière, M., Kale, M.S., Love, J., et al.: Gemma: Open models based on gemini research and technology. arXiv preprint arXiv:2403.08295 (2024)
9. Zhai, X., Mustafa, B., Kolesnikov, A., Beyer, L.: Sigmoid loss for language image pre-training. In: Proceedings of the IEEE/CVF international conference on computer vision. pp. 11975–11986 (2023)